

Tecnologias Escaláveis para Análise de Dados

Scala #1

Mestrado em Engenharia Informática
2023/2024

_introdução

- O que faz de uma linguagem boa para análise e processamento de dados?

_introdução

- Existem atualmente muitas linguagens de programação
- Com (quase) todas elas é possível implementar projetos de análise de dados
- Umas facilitam mais essa tarefa que outras
 - Expressividade da linguagem
 - Nível de abstração
 - Paralelismo
 - Eficiência
 - Suporte para estatística, processamento de ficheiros, conectores para BDs,...
 - ...



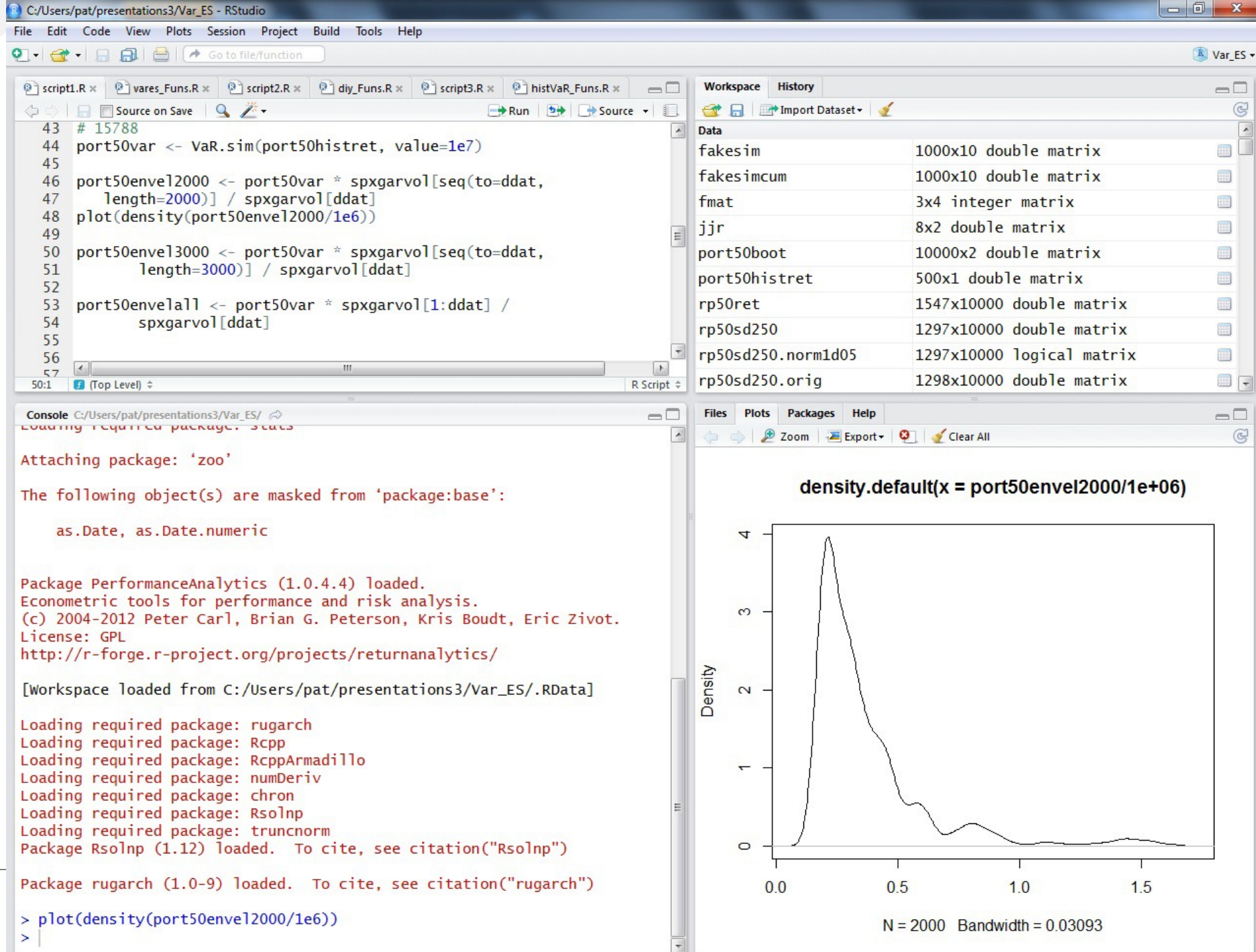
_introdução

- Linguagem de programação de alto nível de âmbito geral
 - Nos últimos anos recebeu várias bibliotecas para análise de dados, estatística, visualização, machine learning
- Tem uma comunidade muito numerosa, com muitas contribuições
- Linguagem muito expressiva e que permite gerar código geralmente mais conciso que outras linguagens
- Suporta programação orientada a objetos e programação funcional
- Tem várias bibliotecas para multi-processamento
- É (provavelmente) a linguagem mais utilizada atualmente para Data Science



_R

- É uma alternativa Open Source a software de estatística que geralmente é muito caro
- Focado em estatística, machine learning e visualização
- Suporta expansão através de bibliotecas
- É possível programar tanto na consola, de forma interativa, como através de scripts
- Possui o conceito de Workspace



_java

- Não é uma linguagem para AD por excelência mas pode ser utilizada
- Não tem as mesmas capacidades de visualização que python ou R, mesmo com bibliotecas externas
- Suporta multiprocessing e programação funcional (a partir da versão 8)

```
final ArrayList<Integer> lista = new ArrayList<>();  
  
List<Integer> positivosVezes10 = lista.stream().filter(x -> x > 0).map( x -> x * 10).collect(Collectors.toList());
```

```
final ArrayList<Integer> lista = new ArrayList<>();  
  
List<Integer> positivosVezes10 = lista.stream().parallel().filter(x -> x > 0).map( x -> x * 10).collect(Collectors.toList());
```



_julia

- Linguagem de elevada performance para computação numérica
- Possui, de base, bibliotecas de álgebra linear, processamento de sinal, visualização, Machine Learning, entre outras
- Suporta execução paralela distribuída
 - Opera em múltiplos processos
 - Tem um mecanismo de message-passing, similar ao utilizado pelo yarn ou zookeeper
- Tem uma comunidade muito forte a desenvolver novas bibliotecas



_scala

• Já lá vamos...

Scala

_introdução

- Diminutivo de Scalable Language
- Integra características de linguagens orientadas a objeto e funcionais
- É uma linguagem Imperativa e Funcional
 - Mas não puramente funcional (como Haskell)
 - Admite variáveis
 - Admite “efeitos colaterais”
 - Objetivo nesta UC: aprender e dominar a vertente funcional
- É compilada para correr na JVM
- Permite integrar código java (classes do SDK ou classes próprias)
- É possível programar de várias formas
 - Scripts, num IDE ou editor de texto, que depois são executados
 - Plugin Scala IntelliJ
 - Scala-IDE (baseado em Eclipse)
 - Notebooks
 - Diretamente na Shell

_documentação

- Documentação disponível em
- <http://www.scala-lang.org/api/current/>

_object-oriented

- Todos os valores são objetos (não existem tipos primitivos)
 - Incluindo os literais
 - É possível invocar métodos sobre um Int



scala

Int

Companion object **Int**

```
abstract final class Int extends AnyVal
```

Int, a 32-bit signed integer (equivalent to Java's `int` primitive type) is a subtype of scala.AnyVal. Instances of Int are not represented by an object in the underlying runtime system.

There is an implicit conversion from scala.Int => `scala.runtime.RichInt` which provides useful non-primitive operations.

_object-oriented

- Tipos e comportamentos de objetos são descritos por classes e *traits*
- Classes são estendidas através de subclassing

```
scala> val valor = 10
valor: Int = 10

scala> valor.
!=    <=    ceil    isInfinity    isWhole    toBinaryString    toOctalString
%     ==    compare    isNaN    longValue    toByte    toRadians
&     >     compareTo    isNegInfinity    max    toChar    toShort
*     >=    doubleValue    isPosInfinity    min    toDegrees    unary_+
+     >>    floatValue    isValidByte    round    toDouble    unary_-
-     >>>    floor    isValidChar    self    toFloat    unary_~
/     ^    getClass    isValidInt    shortValue    toHexString    underlying
<     abs    intValue    isValidLong    signum    toInt    until
<<    byteValue    isInfinite    isValidShort    to    toLong    |

scala> valor. Writable SmartInsert 13 393M of
```

Na prática não existem operadores pois tudo são objetos/métodos: todas as operações são chamadas a métodos

```
scala> 10.+(3)
res2: Int = 13
```

_object-oriented

- Classe
 - É o template que define os comportamentos/estados que um objeto deste tipo suporta
- Trait
 - Equivalente a uma interface em Java
- Object
 - Classe singleton (classe que tem exatamente uma instância)
- Objetos
 - Instâncias de uma classe
 - Têm comportamentos e estados
- Método
 - Representa um comportamento
 - Executam as ações do objeto
- Campos/atributos
 - Representam as características e o estado do objeto

_object-oriented

```
trait Falador{  
  def fala(texto:String):String  
}
```

```
class Person(  
  firstName: String,  
  lastName: String,  
  age: Int) extends Falador  
{  
  
  def this(firstName:String){  
    this(firstName, "", 0)  
  }  
  
  override def toString: String = {  
    if (age > 0)  
      firstName+" "+lastName+", "+age+" anos"  
    else  
      firstName+" "+lastName  
  }  
  
  override def fala(texto: String): String = firstName+": "+texto  
}
```

```
object Teste{  
  
  def main(args: Array[String]): Unit = {  
    val p1 = new Person("Joao", "Costa", 23)  
    val p2 = new Person("Manuel")  
  
    println(p1.fala("Hello World"))  
    println(p1.toString)  
  }  
}
```


_functional

- Todas as funções são vistas como um valor
 - E como todos os valores são objetos, todas as funções são objetos
- Permite a definição de funções anónimas
- Suporta funções de ordem superior
 - Função em que pelo menos um dos parâmetros é uma função

```
def positivo(x : Int) : Boolean = {  
  | x > 0  
}
```

```
val list = List(1,2,3,-4,-2,3) // : List[Int] = List(1, 2, 3, -4, -2, 3)
```

```
list.filter(positivo) // : List[Int] = List(1, 2, 3, 3)
```

_functional

- No paradigma funcional uma função não deve ter “efeitos colaterais” (*side effects*)
 - Não deve alterar qualquer valor fora do contexto da própria função
 - Deve trabalhar apenas sobre os argumentos recebidos e retornar o resultado
- Uma função sem tipo de retorno (`Unit`, equivalente a `void` em Java) é tipicamente uma função que não segue o paradigma funcional
 - Se não retorna nada é porque deve ter efeitos em outras variáveis fora do contexto da função
 - Se não tem, então não faz nada útil
- Escrever na consola é um “efeito colateral”
 - A versão puramente funcional seria formatar o output pretendido e devolvê-lo

_functional

- Um programador com background imperativo geralmente programa utilizando variáveis (`var`)
- Um programador com background funcional geralmente programa utilizando constantes (`val`)
 - Um bom exercício para passar ao paradigma funcional é programar sem utilizar `var` (apesar de este não ser um bom exemplo...)

Imperativo:

```
def printArgs(args: Array[String]): Unit = {  
  var i = 0  
  while (i < args.length) {  
    println(args(i))  
    i += 1  
  }  
}
```

(+/-) Funcional:

```
def printArgs(args: Array[String]): Unit = {  
  for (arg <- args)  
    println(arg)  
}  
  
def printArgs(args: Array[String]): Unit = {  
  args.foreach(println)  
}
```

_statically typed with type inference

- Statically typed: o tipo de cada variável é conhecido no momento da compilação
- Type inference: o tipo é deduzido automaticamente antes de compilar
- Vantagem: código menos verboso mas com a segurança das linguagens tipadas

```
val x = 1 + 2.0
```

```
val y:Int = 1 + 2.0
```

_variáveis vs. constantes

- Não é necessário declarar o tipo, que é inferido pela linguagem
- No entanto é possível, se pretendido, declará-lo
 - Ajuda na documentação do código
 - Permite especificar um tipo quando necessário

```
scala> var nome = "Davide"  
nome: String = Davide
```

Declaração de variável, inferência de tipo

```
scala> val idade = 31  
idade: Int = 31
```

Declaração de constante (equivalente ao *final*, em Java), inferência de tipo

```
scala> idade = idade + 1  
<console>:12: error: reassignment to val  
      idade = idade + 1  
      ^  
    Hello
```

```
scala> nome = nome + " Carneiro"  
nome: String = Davide Carneiro
```

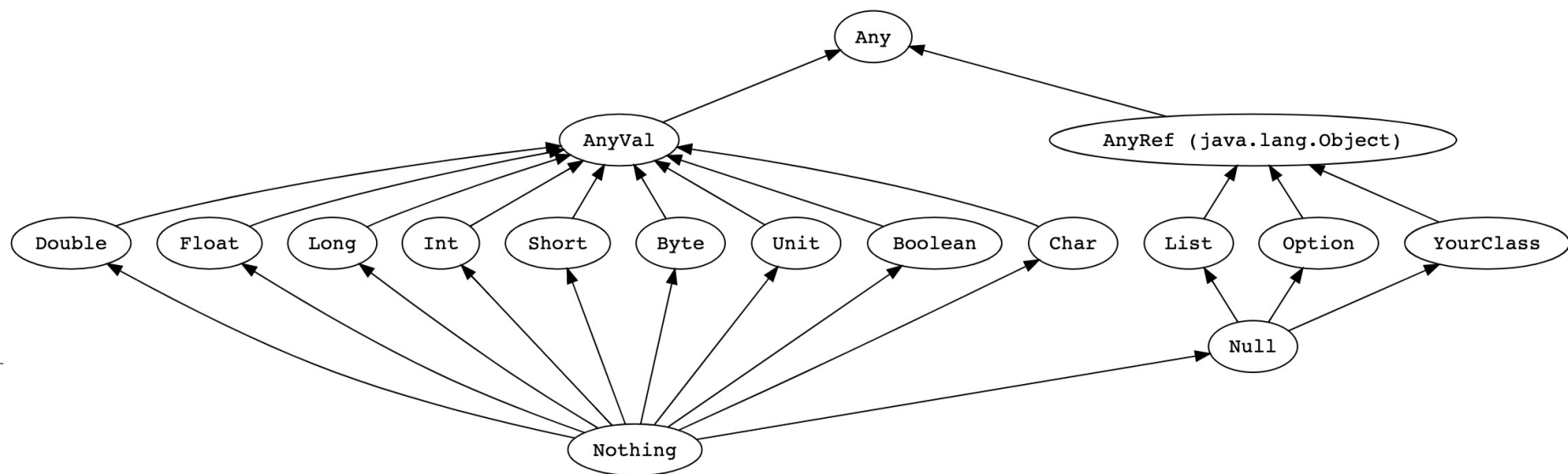
```
scala> var nome: String = "Davide"  
nome: String = Davide
```

```
scala> var idade: Double = 31.2  
idade: Double = 31.2
```

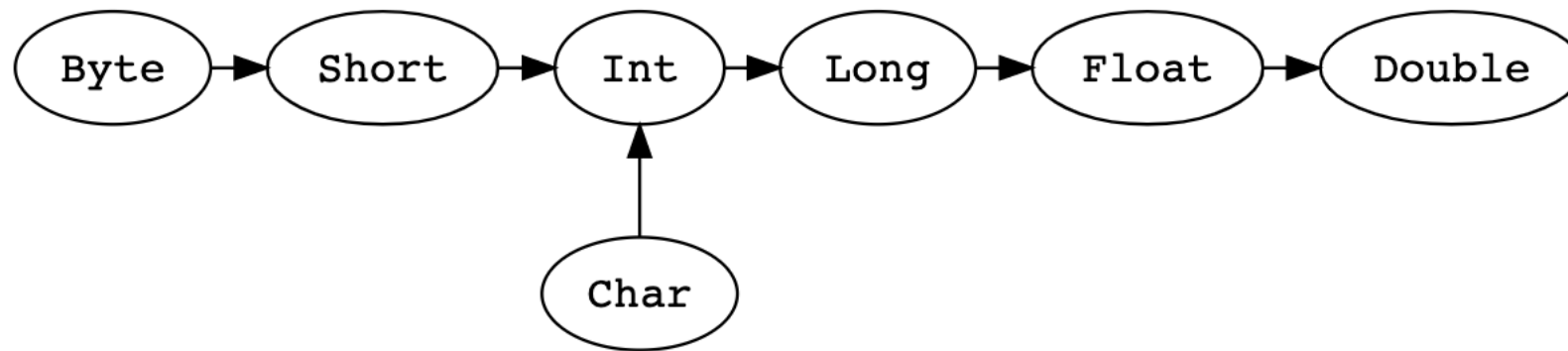
```
scala> var idade: Float = 31.2f  
idade: Float = 31.2
```

_types

- AnyVal – representa todos os tipos para valores
 - Unit – equivalente ao void em Java
- AnyRef – representa todos os tipos com referência
 - Equivalente ao Object em Java
 - Todos os tipos definidos pelo utilizador herdam de AnyRef
- Nothing – subtipo de todos os tipos. Nenhum valor tem tipo Nothing
 - Utilizado, por exemplo, para sinalizar uma não-terminação (e.g. loop infinito, função que não retorna)
- Null – equivalente ao null em Java



_type casting



```
val x: Long = 987654321
val y: Float = x // 9.8765434E8
val z: Long = y // Does not conform
```

_outros tipos de dados

```
scala> var grande = 123456789123456789
<console>:1: error: integer number too large
var grande = 123456789123456789
      ^
```

```
scala> val grande = BigInt("123456789123456789")
grande: scala.math.BigInt = 123456789123456789
```

```
scala> val grande = BigInt(123456789123456789)
<console>:1: error: integer number too large
val grande = BigInt(123456789123456789)
                  ^
```

```
scala> val grande = BigInt(1234)
grande: scala.math.BigInt = 1234
```

```
scala> val grande = BigInt("123456789123456789.1")
java.lang.NumberFormatException: For input string: "3456789.1"
    at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.lang.Integer.parseInt(Integer.java:580)
    at java.math.BigInteger.<init>(BigInteger.java:479)
    at java.math.BigInteger.<init>(BigInteger.java:606)
    at scala.math.BigInt$.apply(BigInt.scala:77)
    ... 32 elided
```

```
scala> val grande = BigDecimal("123456789123456789.1")
grande: scala.math.BigDecimal = 123456789123456789.1
```

```
scala> BigDecimal("123456789123456789") * BigDecimal("123456789123456789.1")
res1: scala.math.BigDecimal = 1.524157878067367852796829966253620E+34
```


_declaração de funções



ção

Java

```
    arg1 + arg2
}
```

Sem result type (Unit, equivalente a void)

```
if (arg1 > arg2)
    println(arg1)
else
    println(arg2)
}
```

_declaração de funções

```
def incrementa(a : Int = 1, inc : Int = 1) : Int = {
  a + inc
}
```

```
println(incrementa(3, 2))
println(incrementa(3))
println(incrementa())
```

Valores por defeito

```
5
4
2
```

```
def HelloWorld() : Unit = {
  println("Hello World")
}
```

```
def HelloWorld2() = {
  println("Hello World")
}
```

Funções sem tipo de retorno

```
def somatorio(nums : Int*) : Int = {
  var soma = 0

  for (num <- nums)
    soma = soma + num

  soma
}
```

```
println(somatorio(1,2,3,4))
println(somatorio(1))
```

Função com número de argumentos variável

_funções parciais

- Há funções que não estão definidas para toda a gama de valores do tipo do parâmetro, chamadas funções parciais:

```
def divide(x: Int, y: Int) : Int = {  
  x / y  
}
```
- Se invocadas com valores não permitidos
 - O programa para
 - Devolve um valor especial
 - Lança uma exceção
- Scala permite definir funções parciais: funções que respondem apenas a um subconjunto dos valores do tipo de dados do parâmetro
- É possível “perguntar” a uma função se consegue lidar com um determinado “valor”
- Especialmente úteis em funções como a `collect` (a ver mais à frente)

_funções parciais

- `apply` define o funcionamento da função para o conjunto de valores para os quais dá resposta
- `isDefinedAt` define a condição que determina os valores para os quais a função não está definida

Tipo(s) de entrada

Tipo(s) de saída

```
val positivoToString = new PartialFunction[Int, String]{  
  def apply(arg : Int) = arg.toString  
  def isDefinedAt(arg : Int) = arg > 0  
}
```

```
val divide = new PartialFunction[(Int, Int), Int]{  
  def apply(arg : (Int, Int)) = arg._1 / arg._2  
  def isDefinedAt(arg : (Int, Int)) = arg._2 != 0  
}
```

```
var x = StdIn.readInt  
var y = StdIn.readInt
```

```
if (divide.isDefinedAt((x, y)))  
  println(divide.apply((x, y)))
```

Uma função parcial pode, na mesma, ser aplicada a um valor para o qual não consegue dar resposta:

```
divide(3, 0)  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
  at Hello$$anon$1.apply(Hello.scala:27)  
  at Hello$$anon$1.apply(Hello.scala:26)  
  at Hello$.main(Hello.scala:54)  
  at Hello.main(Hello.scala)
```

_import

```
scala> abs(-8)
<console>:12: error: not found: value abs
    abs(-8)
    ^

scala> import scala.math._
import scala.math._

scala> abs(-8)
res7: Int = 8
```

Equivalente ao * em java

<https://www.scala-lang.org/api/current/scala/math/index.html>

Pseudo Random Number Generation

```
def random(): Double
  Returns a Double value with a positive sign, greater than or equal to 0.0
  and less than 1.0.
```

Importar funções específicas:

```
scala> import scala.math.{abs, pow}
import scala.math.{abs, pow}
```

Métodos sem parâmetros

Ex.: gerar nº aleatório entre 1 e 10

```
scala> random
res10: Double = 0.3847450693609127

scala> (random * 10 + 1)
res11: Double = 7.475874951787848

scala> (random * 10 + 1).toInt
res12: Int = 1
```

_conditions

```
object Hello {  
  def main(args: Array[String])  
  {  
    val nota = 15  
  
    if(nota < 10)  
      println("Reprovou")  
    else{  
      if (nota < 15)  
        println("Não está mau")  
      else  
        println("Ótimo!")  
    }  
  }  
}
```

_loops

```
object Hello {
  def main(args: Array[String]) {
    val limite = 10
    var i = 0

    while(i < limite)
    {
      println("Hello World")
      i += 1
    }
  }
}
```

```
object Hello {
  def main(args: Array[String]) {
    val lista = List(1,2,3,4,5)

    for (i <- 0 until lista.length ; j <- 0 until lista.length)
      println(i.*(j))
  }
}
```

```
object Hello {
  def main(args: Array[String]) {
    val limite = 10
    var i = 0

    do
    {
      println("Hello World")
      i += 1
    }
    while(i < limite)
  }
}
```

```
object Hello {
  def main(args: Array[String]) {
    val limite = 10
    var i = 0

    for (i<-0 to limite)
      println(i)
  }
}
```

0
1
2
3
4
5
6
7
8
9
10

```
object Hello {
  def main(args: Array[String]) {
    val limite = 10
    var i = 0

    for (i<-0 until limite)
      println(i)
  }
}
```

0
1
2
3
4
5
6
7
8
9

_foreach

```
object Hello {  
  def main(args: Array[String])  
  {  
    val hello = "Hello World"  
  
    hello.foreach(argumento => println(argumento.toUpperCase))  
  
    var i = 0  
    for(i <- 0 until hello.length())  
      println(hello(i).toUpperCase)  
  }  
}
```

foreach recebe como argumento uma função
Neste caso passamos uma função anónima, com um argumento

_function literal (funções anônimas)

Sintaxe:

- (arg1:tipo1, arg2:tipo2,...) => corpo da função

`hello.foreach(argumento => println(argumento.toUpperCase))`

Versão "tradicional"

```
object Hello {  
  def main(args: Array[String])  
  {  
    val hello = "Hello World"  
  
    var i = 0  
    for(i <- 0 until hello.length())  
      println(hello(i).toUpperCase)  
  }  
}
```

Pode-lhes ainda ser atribuído um nome associando-as a uma variável

```
def filter_function(x:Int) : Boolean = {  
  x > 0  
}  
  
val filter_anon = (x:Int) => x > 0 // : Function1[Int, Boolean] = repl.MdocSession$MdocApp$$Lambda$8104/0x0000000802155040@3...  
  
val list = List(1,-1,2,-2,3,-4) // : List[Int] = List(1, -1, 2, -2, 3, -4)  
  
list.filter(filter_function) // : List[Int] = List(1, 2, 3)  
list.filter(filter_anon) // : List[Int] = List(1, 2, 3)  
list.filter(x => x > 0) // : List[Int] = List(1, 2, 3)
```

_function literal in Java

- Java também suporta funções anônimas (entre outros...) a partir da versão 8, bem como uma API de streams

```
public static void main(String[] args)
{
    ArrayList<Character> hello = new ArrayList<>();

    hello.stream().forEach(arg -> System.out.println(Character.toUpperCase(arg)));
}
```

_create object instance

- Objetos são instanciados com a palavra reservada **new**
- Uma nova instância pode ser parametrizada
 - Com valores, entre parênteses curvos
 - Com tipos, entre parênteses retos
- Quando é parametrizada com valor e tipo, o tipo vem primeiro

```
val array = new Array[String](10)
```

_tuples

- Estrutura imutável que pode combinar elementos de diferentes tipos
- São úteis, por exemplo, para o desenvolvimento de métodos que devolvem múltiplos objetos
 - Em Java teríamos que criar uma nova classe para o tipo de retorno
- É possível criar tuplos com até 22 elementos

```
val tuplo = (1,2,3,"coiso")
```

```
println(tuplo._1)  
println(tuplo._2)  
println(tuplo._3)  
println(tuplo._4)
```

Índices começam em 1 e não em 0

```
tuplo.productIterator.foreach(println)
```

Tecnologias Escaláveis para Análise de Dados

Scala #1

Mestrado em Engenharia Informática
2023/2024